AgriSmart

Project Documentation

A voice-first, multilingual agriculture assistant for Indian farmers: mobile app, web, admin dashboard, machine-learning models and a phone-call line.

Item	Detail
Project owner	Sunkireddy Barath
Document version	1.0, 20 October 2026
Scope	Problem, solution, novelty, architecture, technology choices with comparisons, database schema, load balancing, machine-learning training and testing, security, and annotated core code
Components	Backend API (FastAPI), farmer app (Expo / React Native), admin website (Next.js), ML training package, deployment files
Evidence base	All numbers come from files and test runs in the project (model metadata, robustness reports, load-test notes, automated tests). Where a number is an estimate or untested, the text says so

Contents

1. Executive summary 4
2. Problem statement 5
3. Proposed solution 6
4. Innovation, novelty and uniqueness 7
5. System architecture 8
5.1 Components 8
5.2 Typical request 9
6. Technology stack and why each was chosen 10
6.1 Backend framework 10
6.2 Database 10
6.3 Data access and migrations 11
6.4 Farmer client 11
6.5 Admin website 11
6.6 Tabular machine learning (crop and yield) 12
6.7 Image models 12
6.8 Model serving runtime 13
6.9 Voice and language services 13
6.10 Load balancer 13
6.11 Authentication 14
6.12 Packaging and hosting 14
7. Data and database schema 15
7.1 Tables by domain 15
7.2 Relationships 16
7.3 Schema design decisions 16
8. API design 17
9. Load balancing, scalability and thread safety 18
9.1 Load-balancing design 18
9.2 Protecting the database 18
9.3 Measured results 18
9.4 Thread safety and concurrency control 19
10. Security design 20
11. Machine learning: data, algorithms, training and testing 21
11.1 Models and datasets 21
11.2 Training and testing splits, and why 21
11.3 Algorithms in detail 22
11.4 Results: reported versus honest 23
11.5 Worst-case behaviour 24
12. Assistant, language and voice design 25

12.1 AgriBrain 25
12.2 Language support 25
12.3 Voice pipeline 25
13. Core code walkthrough 26
14. Testing and quality assurance 31
15. Deployment and operation 32
16. Limitations and future work 33
16.1 Known limitations 33
16.2 Recommended next steps 33
17. Conclusion 34
Appendix A. Project folder map 35
Appendix B. Glossary 35

1. Executive summary

AgriSmart is an agriculture platform designed around one idea: a farmer should be able to get useful, trustworthy help by **speaking or typing in their own language**, on a basic phone, with little reading and no training. It answers questions about weather, mandi (market) prices, government schemes, crop choice, expected yield, plant diseases and pests, and sends alerts. The same assistant is reachable from a smartphone app, a website, and a plain voice call from a button phone.

- **Who it is for:** small and marginal farmers in India who speak 11 languages (English, Hindi, Telugu, Tamil, Kannada, Malayalam, Marathi, Bengali, Gujarati, Punjabi, Odia) and may not read English fluently.
- **What is built:** a FastAPI backend with PostgreSQL (30 tables), an Expo/React Native app that runs in Expo Go, as an Android APK and on the web, a Next.js admin dashboard, four trained models plus a photo-quality gate, and a load-balanced deployment recipe.
- **What is distinctive:** honest AI (it says "not sure" instead of guessing, and the worst-case accuracy is published), one assistant for three channels, local knowledge blended into model output, and free-by-default sign-in with passwords.
- **Verified today:** 234 backend tests and 6 ML tests pass; the live API passed typed-question tests in all 11 languages (33 of 33) and voice-command tests (17 of 17 that could be run); load tests ran with 0% errors up to 1,000 simulated farmers.
- **Honest limits:** the image models are strong on clean photos and weak on blurry or field photos; the yield model barely beats a simple lookup; free Google speech output allows only 10 requests per day, so the phone's own voice is the free fallback.

2. Problem statement

Indian farmers make high-stakes decisions (what to plant, when to spray, where to sell, which scheme to apply for) with limited, delayed or unreliable information. Existing digital tools fail them in specific, repeatable ways.

Problem	What goes wrong today	Why it matters
Language and literacy	Most apps are English-first or Hindi-first with text menus. Many farmers speak a regional language and read little.	A tool that cannot be used by speaking is not used.
Basic phones	A large share of farmers still use button phones with no data.	Smartphone-only apps exclude them entirely.
Late or wrong pest and disease advice	Farmers wait days for an extension officer or guess from neighbours.	A few days of delay can lose a crop; the wrong spray wastes money and harms soil.
Scattered market information	Mandi prices sit on government portals in tables, in English.	Farmers sell at a disadvantage without a price reference.
Scheme awareness	Hundreds of central and state schemes, each with its own eligibility.	Eligible farmers miss benefits they never heard about.
Over-confident AI	Plant-disease demos report 99% accuracy on tidy photos, then fail on blurry field photos, and still answer confidently.	A confidently wrong diagnosis is worse than no diagnosis.
Paid or fragile sign-in	SMS one-time codes need a paid gateway and fail on poor networks.	Cost and failures block adoption and small deployments.

Problem statement in one sentence: build a free-to-run, multilingual, voice-capable agriculture assistant that gives farmers local, honest and actionable answers on any phone, and that admins can run and monitor at scale.

3. Proposed solution

AgriSmart combines five layers into one product:

1. **One assistant ("AgriBrain")** that understands a question in 11 languages, picks the right tool (weather, prices, schemes, crop advice, yield, disease, pest, calendar) and answers in the farmer's language. Most answers come from fast rules and live data; an optional language model handles open conversation.
2. **Three ways in:** touchscreen app (Expo Go, APK or web), voice inside the app, and a phone call from any handset through a Twilio number.
3. **Four ML models with safety checks:** crop recommendation, yield forecast with a range, plant-disease photo check, pest photo check, plus a gate that rejects photos that are not a leaf or insect.
4. **Live local data:** weather (Open-Meteo), mandi prices (Agmarknet, with data.gov.in as backup), government schemes filtered to the farmer's eligibility.
5. **An operations side:** an admin dashboard for farmers, prices, schemes, support chat, notifications, model versions, voice sessions and system health, plus a worker process for alerts.

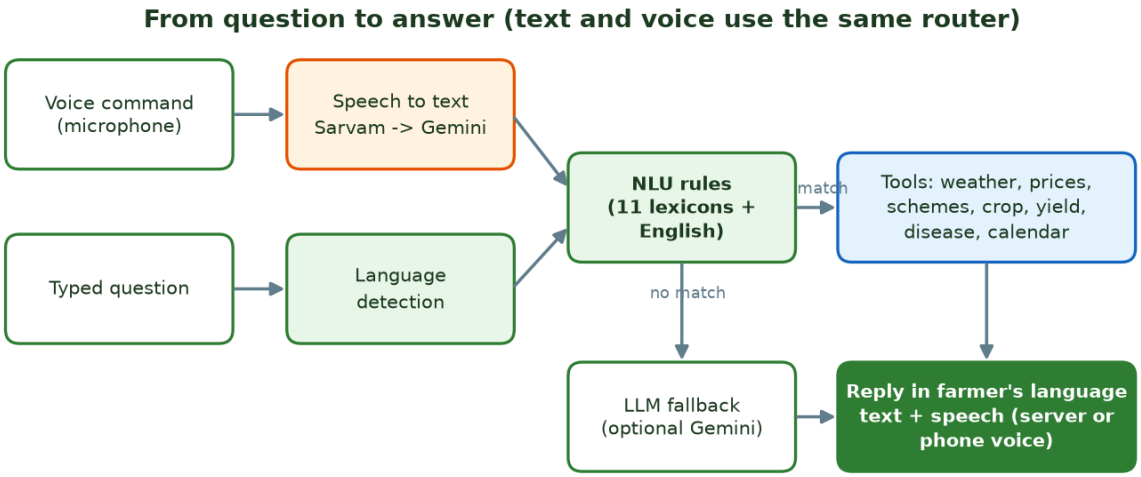


Figure 1. From question to answer. Text and voice share one router.

4. Innovation, novelty and uniqueness

Individually, speech recognition, crop models and chatbots exist. The novelty is in how they are combined and in the engineering choices that make them trustworthy for this audience.

Innovation	What it does	Why it is unusual
Honest confidence gating	Image models answer only above a measured confidence; a separate subject-gate model first checks the photo is a leaf or insect; every diagnosis is stored as accepted or uncertain.	Most demos report one accuracy figure and always answer. Here the gate cut cross-model false acceptance from 33% to 2% on the test sets.
Published worst-case results	Accuracy is reported under 14 kinds of photo damage and for off-topic inputs, and models are re-tested on unseen conditions.	Reveals that 99.8% (clean) becomes 4.7% (heavy blur), so the product can be designed around the truth.
Local knowledge fused with ML	Crop score = model likelihood x regional prior (area grown in the farmer's state and season) x agronomic rules.	A soil-only classifier ignores what actually grows locally. This reduces nonsense recommendations.
One brain, three channels	App, in-app voice and a button-phone call all use the same router and tools.	Farmers without smartphones get the same answers as those with them.
Rules first, language model second	Deterministic NLU with per-language lexicons handles most questions; an LLM is called only when rules fail, and can be switched off.	Fast, free, predictable, and works offline from the LLM provider.
Code-mixed language handling	The English lexicon is always applied with the farmer's language.	Farmers mix English words ("onion price") into their language.
Resilient voice chain	Sarvam, then Gemini, then the phone's own voice, chosen per task; recognition tries the fast model first.	A provider running out of credits or stalling does not break the experience.
Free-by-default accounts	Password sign-in stored as Argon2id hashes, with lockout. SMS or e-mail codes are optional.	No paid gateway is needed to run a real deployment.
Cost-aware ML serving	Models are exported to ONNX, one disease model compressed to int8 (4.9 MB), XGBoost pinned to one thread per request.	Cut CPU per prediction about 4x and lets a single CPU server serve real traffic.
Measured, not assumed, scaling	Load-tested through a load balancer; connection pools budgeted against the database limit; overload returns a clean 503 with Retry-After.	Many student projects never test beyond one user.

5. System architecture

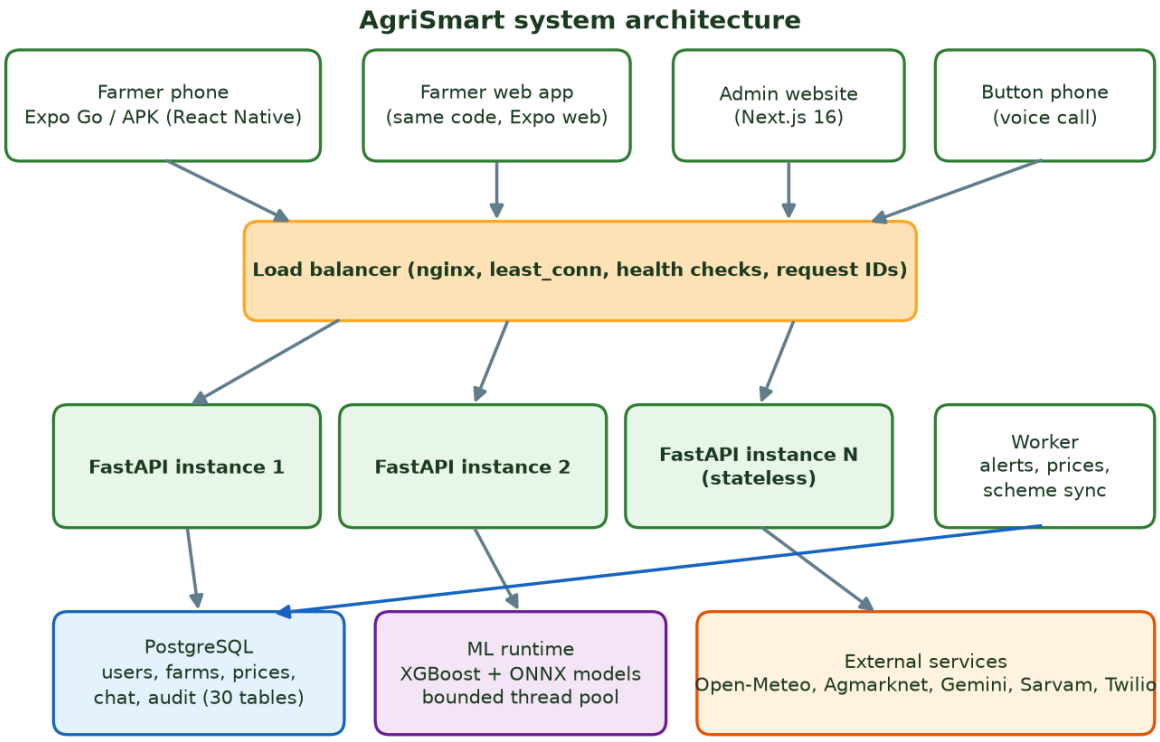


Figure 2. Components and data flow.

5.1 Components

Component	Technology	Responsibility
Farmer app	Expo SDK 57, React Native 0.86, Expo Router, TanStack Query, i18next	Screens, voice recording and playback, language choice, secure token storage; runs on Android, iOS (Expo Go) and web from one codebase
Admin website	Next.js 16, React 19	Backend-for-frontend: stores the admin's tokens in httpOnly cookies so they never reach browser JavaScript
API	Python 3.12, FastAPI, SQLAlchemy (async), Alembic, Pydantic	Authentication, business logic, ML inference, live data, assistant routing, voice
Database	PostgreSQL 16	All durable state: accounts, farms, predictions, chat, prices, audit log
Worker	Python process (same codebase)	Alerts (weather, price, outbreak, calendar), notification delivery, price refresh, scheme sync, translation warm-up
ML runtime	XGBoost, ONNX Runtime, NumPy, Pillow	Loads versioned model artifacts; predictions on a bounded thread pool
Load balancer	nginx (Docker), Caddy used for local benchmarks	Spreads requests over stateless API instances, health-based routing, edge limits
External services	Open-Meteo, Agmarknet, data.gov.in, Gemini, Sarvam, Twilio, mySchemе	Weather, prices, language and voice, telephony, schemes

5.2 Typical request

1. The app sends `POST /api/v1/assistant/query` with a bearer token.
2. nginx forwards it to the least busy API instance.
3. The API checks the token, applies the per-user rate limit, loads the farmer's context (language, location, farms).
4. AgriBrain runs the NLU rules, picks a tool, which reads the database or a cache or calls a live source or an ML model on the thread pool.
5. The reply is rendered in the farmer's language, saved to chat history with its latency, and returned. For voice, the reply is also synthesised to audio.

6. Technology stack and why each was chosen

For each layer the table shows the chosen technology first, then the alternatives that were considered and the trade-off. Where the project measured something, the text says so; otherwise the comparison is design reasoning based on the project's constraints: a small team, free-to-run hosting, a CPU-only server, Indian-language voice, and farmers on weak networks.

6.1 Backend framework

The API must handle many concurrent I/O-heavy calls (database, speech, weather) and serve CPU-bound ML.

Option	Strengths	Why chosen or not chosen
FastAPI (chosen)	Async by default, automatic validation and OpenAPI docs, Python (same language as the ML code), very fast to develop.	Best fit: one language for API and ML, async I/O for slow external calls, interactive docs for testing.
Django REST Framework	Batteries included, admin site, mature.	Heavier and synchronous by default; async support is newer. The custom admin dashboard replaces Django admin anyway.
Flask	Minimal, simple.	No built-in validation or async model; more glue code to reach the same result.
Node.js (Express / NestJS)	Great async performance, one language with the front end.	ML models and data code live in Python; two languages would be needed.
Spring Boot (Java)	Strong typing, enterprise tooling.	Slower to build, heavier to host for free, no direct ML reuse.

> **Decision:** FastAPI, because the product is ML-centred and I/O-bound, and the team needed the shortest path from model to endpoint.

6.2 Database

Data is relational (farmers, farms, crop cycles, prices) with some flexible documents (model metrics, notification preferences).

Option	Strengths	Why chosen or not chosen
PostgreSQL 16 (chosen)	ACID transactions, row locks (`FOR UPDATE SKIP LOCKED`), JSON columns, strong indexes, free everywhere (Supabase free tier).	Row locks make one-time codes and refresh tokens safe under concurrency, and `SKIP LOCKED` lets several workers share a queue. JSON handles flexible fields.
MySQL / MariaDB	Popular, simple.	Weaker JSON and locking features for the queue and token logic.
MongoDB	Flexible documents.	Relationships between farmers, farms and cycles are core; joins and constraints matter. Transactions are less central there.
SQLite	Zero setup.	Used for the automated tests, but not for many concurrent writers.
Firebase / Firestore	Fast start, real time.	Vendor lock-in, harder complex queries and audits, and cost grows with reads.

> **Decision:** PostgreSQL, because correctness under concurrency (tokens, codes, queues) is a requirement, not an option.

6.3 Data access and migrations

Schema changes must be repeatable between a laptop and a server.

Option	Strengths	Why chosen or not chosen
SQLAlchemy 2 (async) + Alembic (chosen)	Typed models, async sessions, versioned migrations.	Matches FastAPI's async model; migrations run the same way in Docker and locally.
Django ORM	Easy.	Tied to Django.
Raw SQL	Full control.	More code, easy to make injection or consistency mistakes.

> **Decision:** SQLAlchemy with Alembic. The project has 8 migrations so far, including the password lockout columns added for this version.

6.4 Farmer client

One codebase must serve Android, iPhone testing and the web, install easily, and use the microphone, camera and location.

Option	Strengths	Why chosen or not chosen
Expo / React Native (chosen)	One TypeScript codebase for Android, iOS and web; Expo Go lets anyone test by scanning a QR code; EAS builds an APK in the cloud.	Fastest route to a real phone test and an APK; same code as the web version.
Flutter	Great UI, one codebase.	Dart is a new language for the team; no instant QR-code testing; the web build is heavier.
Native Kotlin / Swift	Best performance and platform features.	Two codebases and far more effort for a small team.
Web app only (PWA)	No install.	Weaker microphone, notifications and offline behaviour on low-end phones.

> **Decision:** Expo / React Native, to reach Android, iOS and the web at once with one team.

6.5 Admin website

Admins need tables, charts, chat and model controls; sign-in must be safe.

Option	Strengths	Why chosen or not chosen
Next.js 16 (chosen)	Server components and server actions act as a backend-for-frontend; httpOnly cookies keep tokens away from browser scripts.	Safer token handling than a pure single-page app, and easy deployment on Vercel or Docker.
React single-page app (Vite)	Simple.	Tokens would have to live in browser storage, which scripts can read.
Angular	Structured.	Heavier, and the team already uses React in the app.

Option	Strengths	Why chosen or not chosen
Django admin / low-code tools	Very fast.	Limited custom screens (model activation, support chat, voice sessions).

Decision: Next.js, mainly for the security of the cookie-based session.

6.6 Tabular machine learning (crop and yield)

Small tables of numbers (2,200 rows for crops, about 10,400 for yield).

Option	Strengths	Why chosen or not chosen
XGBoost (chosen)	Excellent on small tables, handles interactions, native quantile regression, fast on CPU, saves to a plain JSON file.	Quantile objective gives the low / likely / high yield range directly; no pickle files, so a model file cannot run code.
Random forest	Robust, few settings.	Slightly less accurate on tabular data and no native quantile output.
Logistic regression / kNN	Simple, transparent.	**Measured:** kNN 98.0% and logistic regression 97.3% against XGBoost 99.3% on the crop data. Close, so XGBoost is not essential for crops, but it gives better probabilities for ranking.
Neural networks (MLP, TabNet)	Flexible.	Overkill for ~2,000 rows, harder to tune and explain.
LightGBM	Fast, similar accuracy.	A reasonable alternative. XGBoost was chosen for its quantile objective and familiarity.

Decision: XGBoost. The comparison above shows honestly that simple models come close on the crop data; the real gains are the probability quality and the yield range.

6.7 Image models

Photos of leaves and insects must be classified on a CPU server and potentially on a phone later.

Option	Strengths	Why chosen or not chosen
MobileNetV3-Large (chosen)	Small (about 17 MB), fast on CPU, strong accuracy after transfer learning.	Fits free-tier CPU hosting and compresses to 4.9 MB with int8.
ResNet-50	Strong baseline.	About 5 times larger and slower on CPU for little gain here.
EfficientNet	Accurate.	Slower to train and serve on CPU than MobileNet at similar accuracy for this task.
Vision Transformer	State of the art with huge data.	Needs far more data and compute than this project has.
Cloud vision API	No training.	Recurring cost, no control over classes, privacy of farm photos.

Decision: MobileNetV3 with transfer learning from ImageNet weights.

6.8 Model serving runtime

Inference must be fast, small and safe to load.

Option	Strengths	Why chosen or not chosen
ONNX Runtime (chosen)	Framework-neutral, fast CPU inference, int8 support, small install.	One runtime for both image models; avoids shipping PyTorch to production.
PyTorch in the API	Simple.	Large dependency and slower start-up.
TensorFlow Lite / TF Serving	Mobile friendly.	Would require a TensorFlow training path.
Triton / dedicated model server	Scales GPU serving.	Over-engineered for a CPU-only deployment.

> **Decision:** ONNX Runtime, with XGBoost's own runtime for the tables.

6.9 Voice and language services

Needs Indian-language speech recognition, natural speech output and translation, on a free or low budget.

Option	Strengths	Why chosen or not chosen
Sarvam AI (primary)	Built for Indian languages, natural voices, male and female.	Paid; ran out of credits during testing. The system moves to the next provider automatically.
Google Gemini (free backup)	Free key; recognised every language tested; also translation.	Free tier limits: only 10 speech-output requests a day per model. The full flash model stalled on audio, so the fast lite model is tried first.
Phone's own text-to-speech	Free, unlimited, offline.	Quality and language coverage depend on the phone. It is the final free fallback.
Bhashini (government)	Free for Indian languages.	Supported in the code; needs registration and has variable availability.
Whisper (local)	No external service.	Recognition only, and CPU-heavy.
Google Cloud / Azure speech	Reliable.	Paid per use.

> **Decision:** A chain: Sarvam, then Gemini, then the phone's voice. Typed answers always work and cost nothing.

6.10 Load balancer

Distribute traffic over stateless API copies and survive an instance failing.

Option	Strengths	Why chosen or not chosen
nginx (chosen for deployment)	Industry standard, `least_conn`, keep-alive to upstream, rate limits, gzip, official Docker image.	Configuration validated with `nginx -t`; traffic spread exactly evenly over three instances in a local run.

Option	Strengths	Why chosen or not chosen
Caddy	Automatic HTTPS, simple config.	**Measured:** used for local Windows benchmarks (p95 71 ms at 100 farmers). nginx for Windows is single-threaded, so it is not a fair benchmark; production uses nginx on Linux.
HAProxy	Excellent L4/L7 balancing.	Equally valid; nginx also serves the admin site and static files.
Traefik	Dynamic Docker discovery.	Heavier to configure for a single-host recipe.
Cloud load balancer	Managed.	Costs money; the project targets free or self-hosted setups.

> **Decision:** nginx with `least_conn`, because inference and speech requests have very uneven durations.

6.11 Authentication

Sign-in must be free, safe and work the same on phone and web.

Option	Strengths	Why chosen or not chosen
Argon2id + JWT + rotating refresh token (chosen)	Argon2id is the current recommended password hash; short-lived access tokens; refresh tokens stored hashed and revocable.	No third-party account, no per-SMS cost, tokens revocable, lockout in the database so it holds across servers.
SMS one-time codes only	No password to remember.	Paid gateway, delivery failures, and a cost per login. Kept as an optional extra.
Firebase Auth / Auth0	Quick to add.	Vendor dependency and cost at scale.
Google sign-in	Familiar.	Many farmers lack or do not use Google accounts; needs a client ID per app.

> **Decision:** Passwords with Argon2id, plus optional codes for password reset.

6.12 Packaging and hosting

One recipe for a laptop, a single server and free tiers.

Option	Strengths	Why chosen or not chosen
Docker Compose (chosen)	Same containers everywhere, scale with `--scale api=N`.	Simple for one host, matches the load-balancing demo.
Kubernetes	Autoscaling, self-healing.	Operational overhead too high for the current size.
Serverless functions	Pay per use.	Cold starts and model loading do not suit ML inference.

> **Decision:** Docker Compose now; Kubernetes is the natural next step if traffic demands it.

7. Data and database schema

The database has **30 tables** in PostgreSQL, created by 8 Alembic migrations. Every table has a UUID primary key and created / updated timestamps (except high-volume cache tables). Flexible fields (model metrics, notification preferences, prediction inputs) use JSON columns; everything that is queried or constrained is a normal column.

7.1 Tables by domain

Domain	Tables and key columns
Identity and security	`users` (role, phone, email, password_hash, is_active, last_login_at, failed_logins, locked_until) `auth_sessions` (user_id, refresh_token_hash, expires_at, revoked_at, user_agent) `otp_codes` (identifier, code_hash, expires_at, attempts, consumed_at) `audit_logs` (actor_user_id, action, entity, details, ip_address)
Farmer and farm	`farmers` (user_id, name, phone, preferred_language, primary_channel, location_id, voice_preferences, notification_preferences, push_token) `locations` (state, district, village, latitude, longitude) `farms` (farmer_id, name, area_acres, soil_type, irrigation) `soil_records` (farm_id, nitrogen, phosphorus, potassium, ph, organic_carbon, tested_on) `crop_cycles` (farm_id, crop_id, season, sowing_date, area_acres, status, actual_yield_quintals) `calendar_events` (farmer_id, crop_cycle_id, event_type, scheduled_on, done)
Knowledge base	`crops` (slug, names, seasons, water_requirement, suitable_soils, yield range, growing_states, calendar_template) `diseases` and `pests` (slug, names, symptoms, treatment, prevention, safety_warnings, verified_by) `government_schemes` (code, names, state, district, summary, benefits, eligibility_text, official_url) `scheme_rules` (scheme_id, rule_type, value) `translation_cache` (source_hash, target_lang, text, provider)
AI and predictions	`ml_models` (name, version, framework, artifact_path, metrics, is_active) `crop_recommendations`, `yield_predictions`, `disease_predictions` (farmer_id, model_version, inputs, result, confidence, accepted, latency_ms)
Market and weather	`markets` (name, state, district) `market_prices` (market_id, commodity, arrival_date, min / max / modal price, source) `weather_cache` (cache_key, payload, expires_at)
Conversation and voice	`chat_sessions` (farmer_id, channel, language) `chat_messages` (session_id, role, text, intent, handled_by, latency_ms) `voice_sessions` (farmer_id, channel, language, provider, duration_seconds, turns, end_reason) `voice_clips` (audio, mime, expires_at)
Notifications and support	`notifications` (idempotency_key, category, title, body, target) `notification_deliveries` (notification_id, farmer_id, channel, status, attempts, sent_at, read_at) `support_messages` (farmer_id, sender, body, admin_user_id, read_at)

7.2 Relationships

Relationship	Type	Meaning
users -> farmers	1 to 1	A farmer account has one profile
users -> auth_sessions	1 to many	One row per signed-in device; revoking a row signs that device out
farmers -> farms	1 to many	A farmer can have several plots
farms -> soil_records, crop_cycles	1 to many	Soil tests over time; one cycle per crop per season
crop_cycles -> crops	many to 1	Which crop is grown
crop_cycles -> calendar_events	1 to many	Tasks generated from the crop template
farmers -> chat_sessions -> chat_messages	1 to many to many	Conversation history with intent and latency per message
farmers -> voice_sessions	1 to many	One row per voice conversation or call
markets -> market_prices	1 to many	Daily prices per market and commodity
government_schemes -> scheme_rules	1 to many	Machine-checkable eligibility rules
notifications -> notification_deliveries	1 to many	One delivery per farmer and channel, claimed with row locks

7.3 Schema design decisions

- **Passwords are never stored**: only `password_hash` (Argon2id). The failed-attempt counter and `locked_until` live in `users` so a lock applies across all API instances.
- **One-time codes and refresh tokens are stored hashed** (`code_hash`, `refresh_token_hash`), so a database leak does not reveal usable secrets.
- **Predictions are stored with inputs, model version and an accepted flag**, which lets admins audit diagnoses and compare model versions.
- **Idempotency keys on notifications** make repeated scheduled jobs safe: the same alert is not sent twice.
- **Caches are tables** (`weather_cache`, `translation_cache`, `voice_clips`) with expiry, so they survive restarts and are shared by all instances.
- **Reference data is dynamic**: crops, diseases, pests and schemes are rows edited by admins (with `verified_by`), not text hard-coded in the app.

8. API design

The API is versioned under `/api/v1`, documented automatically at `/docs` in development (hidden in production), and returns errors in one consistent format (`code`, `message`, `details`, `request_id`).

Group	Endpoints
Authentication	`POST /auth/register`, `/auth/password-login`, `/auth/admin/login`, `/auth/refresh`, `/auth/logout`, `/auth/otp`, `/auth/login` (code), `/auth/password-reset`, `PUT /auth/password`, `GET /auth/methods`
Farmer and farms	`GET/PUT /farmer/profile`, `PUT /farmer/push-token`, `GET/POST /farms`, `PUT /farms/{id}`, `GET/POST /farms/{id}/soil`, `POST /farms/{id}/cycles`, `GET /farms/cycles`, `PATCH /farms/cycles/{id}`
AI features	`POST /crop/recommend`, `GET /crop/catalog`, `POST /yield/predict`, `POST /disease/predict`, `POST /pest/predict`, `POST /profit/calculate`
Assistant and voice	`POST /assistant/query`, `GET /assistant/sessions/{id}/messages`, `GET /voice/capabilities`, `GET /voice/voices`, `POST /voice/tts`, `POST /voice/session`, `POST /voice/session/{id}/utterance`, `POST /voice/session/{id}/end`, `POST /voice/telephony/twilio/incoming`
Live data	`GET /weather`, `GET /market/prices`, `GET /market/history`, `GET /schemes`, `GET /schemes/eligible`, `GET /calendar`, `PATCH /calendar/events/{id}`
Alerts and support	`GET /notifications`, `POST /notifications/read`, `GET/POST /support`, `GET /support/unread`, `POST /support/read`
Admin	`/admin/analytics`, `/farmers`, `/farms`, `/crops`, `/knowledge/{kind}`, `/markets/prices`, `/markets/import`, `/schemes`, `/models`, `/notifications`, `/support/threads`, `/voice-sessions`, `/weather/cache`, `/system`
Operations	`GET /health` (process alive), `GET /ready` (database reachable), `/metrics` (blocked at the edge)

9. Load balancing, scalability and thread safety

9.1 Load-balancing design

The API is **stateless**: no per-user data lives in process memory except caches and the in-process rate limiter. Any instance can serve any request, so capacity grows by running more copies behind the balancer.

```
upstream api {
    least_conn;                       # send each request to the least busy instance
    zone api_upstream 64k;
    server api:8000 resolve max_fails=3 fail_timeout=10s;
    keepalive 64;                     # reuse upstream connections
}
limit_req_zone $binary_remote_addr zone=per_ip:20m rate=200r/s;   # coarse flood guard
```

`least_conn` is used instead of round-robin because requests differ greatly in length: a typed weather question takes milliseconds, a voice turn can take several seconds. Round-robin can pile slow requests onto one instance. `resolve` re-reads Docker DNS so `docker compose up --scale api=4` takes effect without restarting nginx. Each request gets one `X-Request-ID` that nginx creates and the API logs and echoes, so a request can be followed end to end.

9.2 Protecting the database

- **Connection budget:** `replicas x workers x (DB_POOL_SIZE + DB_MAX_OVERFLOW)` must stay below PostgreSQL `max_connections`. The Compose file budgets 4 x 2 x 15 = 120 for 200 allowed connections.
- **Fast failure:** if the pool is exhausted for 10 seconds the API returns **503 `service_busy` with `Retry-After`**, instead of hanging or returning a 500.
- **Bounded ML work:** inference runs on a thread pool capped at `ML_MAX_CONCURRENCY`; extra requests wait instead of overloading the CPU.

9.3 Measured results

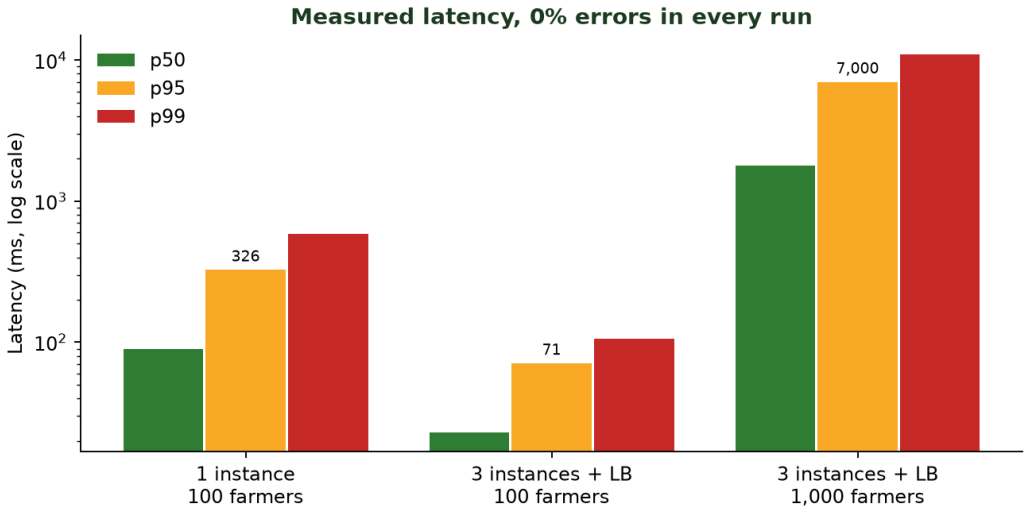


Figure 3. Latency from the load test (development laptop, 2 October 2026).

Setup	Farmers	Requests	Errors	p50	p95	p99
One API instance	100	2,696	0%	90 ms	326 ms	584 ms
Caddy, least_conn, 3 instances	100	4,409	0%	23 ms	71 ms	106 ms

Setup	Farmers	Requests	Errors	p50	p95	p99
Caddy, 3 instances, 4 generators	1,000	13,182	0%	1.8 s	7 s	11 s

> **Reading the numbers:** at 1,000 farmers the laptop itself is saturated (about 81% CPU, 3.8 cores spent on the load generators). Nothing failed; requests queued and were answered. One worker handles about 45 requests per second, roughly 55 active farmers at low latency. These are laptop figures, not a production promise.

9.4 Thread safety and concurrency control

Risk	How it is handled
Two requests use the same one-time code	`SELECT ... FOR UPDATE` on the code row; the code is marked consumed in the same transaction
Refresh token used twice	Row lock on the session; the old token is revoked as the new one is issued (rotation)
Several workers send the same alert	Deliveries claimed with `FOR UPDATE SKIP LOCKED`; alerts carry idempotency keys
Model loaded twice under load	One `asyncio.Lock` per model in the registry
Rate-limit table corrupted by threads	A `threading.Lock` guards the sliding-window table; memory is capped at 100,000 keys
CPU oversubscription	`anyio.CapacityLimiter` caps parallel inference; XGBoost pinned to one thread per request
Hammering upstream services	Semaphores cap parallel calls to Sarvam, Agmarknet and mySchemе and the number of simultaneous phone calls
Brute-force password guessing	Failure counter and lock stored in PostgreSQL, so every API instance sees the same state

10. Security design

Area	Measure
Passwords	Argon2id hashing; minimum 8 characters; a dummy hash is verified for unknown accounts so the time taken does not reveal which accounts exist
Brute force	5 wrong passwords lock the account for 15 minutes; per-IP limit of 10 sign-ins per minute
Tokens	Access token 30 minutes (HS256); refresh token 30 days, rotating, stored hashed, revocable; reset password revokes all sessions
Authorisation	Role checks on every route; a farmer token is refused on admin routes (403)
Rate limits	Sliding window per user or per IP: 60 per minute general, 20 per minute assistant, 5 per minute admin login
Input validation	Pydantic on every request; phone numbers normalised to +91XXXXXXXXXX
Uploads	Only JPEG, PNG and WebP; size limits; decompression-bomb guard (40 megapixels); photo-quality gate
Models	Framework-native files only (XGBoost JSON, ONNX), never pickles, so loading a model cannot run code
Telephony	Twilio signatures verified; media stream tokens signed; audio links signed and expiring
Headers	`nosniff`, `X-Frame-Options: DENY`, `no-referrer`, HSTS
Secrets	Environment variables only; `.env` is git-ignored; the mobile app holds no third-party keys
Device storage	Tokens in the OS secure store on phones; the web build uses browser storage because browsers have no keystore
Audit	Admin actions are written to `audit_logs` with IP and request ID

> **Known trade-off:** with password-only registration nobody proves they own a phone number. The first person to register a number owns that account. For deployments where this matters, enable SMS or e-mail codes and require one at registration.

11. Machine learning: data, algorithms, training and testing

11.1 Models and datasets

Model	Task and data	Algorithm	Output
Crop recommendation (crop_v3)	22 crops; 2,200 rows of N, P, K, temperature, humidity, pH, rainfall (Kaggle)	XGBoost multi-class classifier, 800 trees, depth 8, learning rate 0.05, column sampling 0.6, L2 = 2	Probability per crop, ranked with regional prior and rules
Yield prediction (yield_v3)	19 crops, 30 states, 1997-2020; about 10,400 rows (Kaggle, India)	XGBoost quantile regression (P10, P50, P90) on log(1 + yield), interval widened by a calibration factor	Low, likely and high yield per hectare
Disease (disease_v1)	38 classes, 14 crops, PlantVillage photos (Kaggle)	MobileNetV3, two-stage transfer learning, exported to ONNX and compressed to int8 (4.9 MB)	Top-3 labels with confidence, accepted or uncertain
Pest (pest_v1)	12 insect classes, 5,494 photos (Kaggle)	MobileNetV3-Large, same recipe, served as fp32 because int8 lost too much accuracy	Top-3 labels with confidence
Subject gate (subject_v1)	Leaf or insect vs other photos (pests + Imagenette), 6,413 images	Small MobileNet binary classifier, ONNX	Reject photos that are not a clear subject

11.2 Training and testing splits, and why

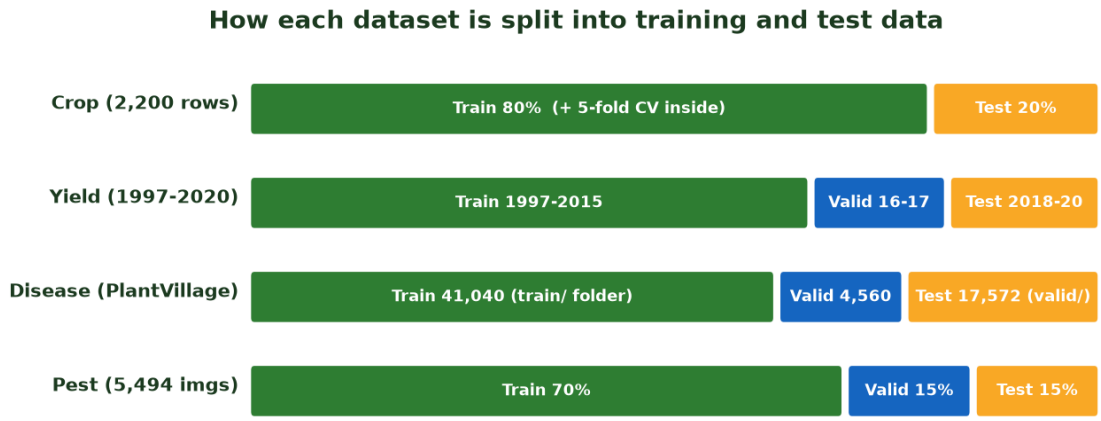


Figure 4. Train, validation and test splits for each model.

Model	Split used	Why this split
Crop	Random 80% train / 20% test, stratified by crop (seed 42). Hyper-parameters chosen by 30-trial random search scored with 5-fold cross-validated log-loss **on the training part only**.	Every crop has exactly 100 rows, so stratifying keeps all classes balanced in both parts. The data has no date or place column, so a time or region split is not possible. Tuning on the training part only keeps the test score an unbiased estimate.

Model	Split used	Why this split
Yield	By **time**: train 1997-2015 (8,235 rows), validation 2016-2017 (1,084), test 2018-2020 (1,110).	Yields trend upward over years and differ by state. A random split lets the model see neighbouring years of the same state and overstate accuracy. Splitting by time mimics real use: train on the past, forecast the future. The validation years drive early stopping and calibrate the P10-P90 band.
Disease	Dataset's own `train/` folder split into train (41,040) and validation (4,560) with stratification; the shipped `valid/` folder (17,572 images) is the test set.	Keeps a clearly separate test set. **Weakness found:** about 62% of test files are rotated or flipped copies of photos that also appear in training. The recommended fix is to split by source photo.
Pest	Stratified 70% / 15% / 15% by class.	A perceptual-hash check found only 4 of 825 test photos with a lookalike in training, so a random image split is acceptable here.
Subject gate	Held-out set from the same mix.	Small binary task; measured by how many real subjects are kept and non-plants rejected.

11.3 Algorithms in detail

Crop recommendation

XGBoost builds many shallow decision trees one after another, each correcting the errors of the previous ones. With 22 crops it uses the `multi:softprob` objective and returns a probability for each crop. The raw probability is then **tempered** (raised to a power below 1, because the synthetic training data makes the model over-confident) and multiplied by two extra factors: the share of the farmer's state and season actually planted with that crop (from government area statistics), and agronomic rule adjustments (irrigation, soil type, rotation, elevation). The product ranks the whole crop catalogue, including crops the model never saw, which get a neutral model score.

Yield forecast

Three XGBoost quantile models predict the 10th, 50th and 90th percentile of the logarithm of yield, using year, rainfall, fertiliser, pesticide, crop, season and state. Training on the log scale handles the huge difference between sugarcane (about 64 t/ha) and moth bean (0.5 t/ha). The raw band covered only 57.8% of real outcomes, so it was widened by a factor of 1.75 using the validation years, raising test coverage to 78.5% (target 80%).

Image models

A MobileNetV3 network pre-trained on ImageNet is adapted in two stages. **Stage 1** trains only the new classification head (2 epochs, learning rate 1e-3). **Stage 2** fine-tunes the whole network at a lower rate (up to 10 epochs, 2e-4) with AdamW, weight decay 1e-4, cosine learning-rate decay, label smoothing 0.1 and early stopping with patience 3. Training photos are randomly cropped, flipped, rotated and colour-jittered so the model tolerates normal variation. The final model is exported to ONNX; the disease model is also compressed to int8.

11.4 Results: reported versus honest

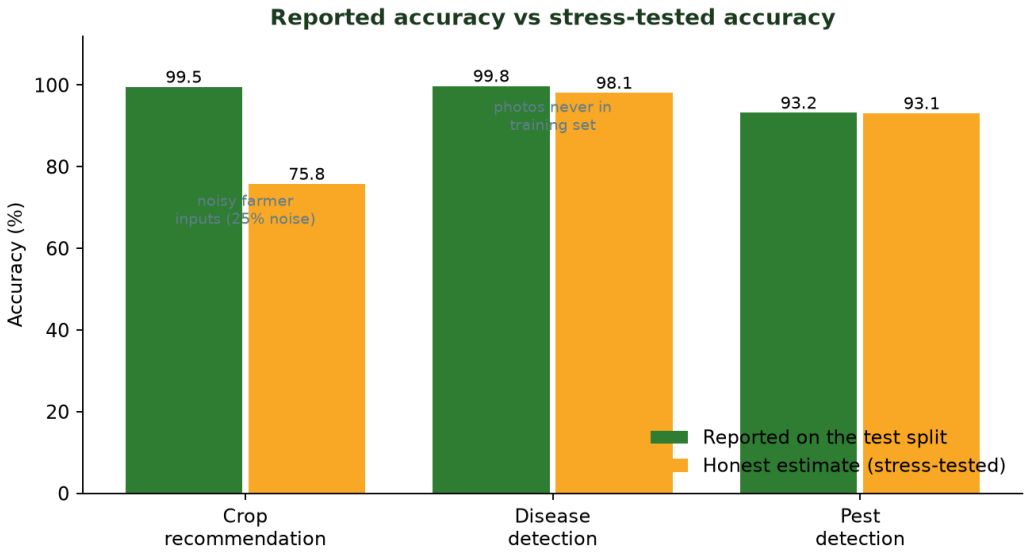


Figure 5. Reported accuracy versus accuracy after stress testing.

Model	Reported	What the stress tests showed
Crop	99.55% accuracy, top-3 100%, 5-fold CV 99.26% +/- 0.43	Easy dataset: kNN reaches 98.0%, logistic regression 97.3%. Noisy inputs hurt badly: noise of 10% of normal spread gives 94.6%, 25% gives 75.8%, 50% gives 52.2%. Nonsense inputs still produce an answer.
Yield	R-squared 0.907, MAE 1.51 t/ha	Median error 14.4% but mean error 21.6% (the stored "MAPE" was the median and has been corrected). A simple "median of that crop in that state" lookup scores R-squared 0.92. Rainfall, fertiliser and pesticide add no measurable skill. On an unseen state the median error is 31%; on an unseen crop 45%.
Disease	99.79% accuracy	98.9% on the served int8 model; 98.1% on photos whose source was never in training. Weakest: tomato leaf mold 89%, potato late blight 90%. Real field photos were not available to test.
Pest	93.21% accuracy	93.1% with near-duplicates removed. At the 0.6 confidence threshold it answers 88.7% of photos and is 97.4% correct on those.

11.5 Worst-case behaviour

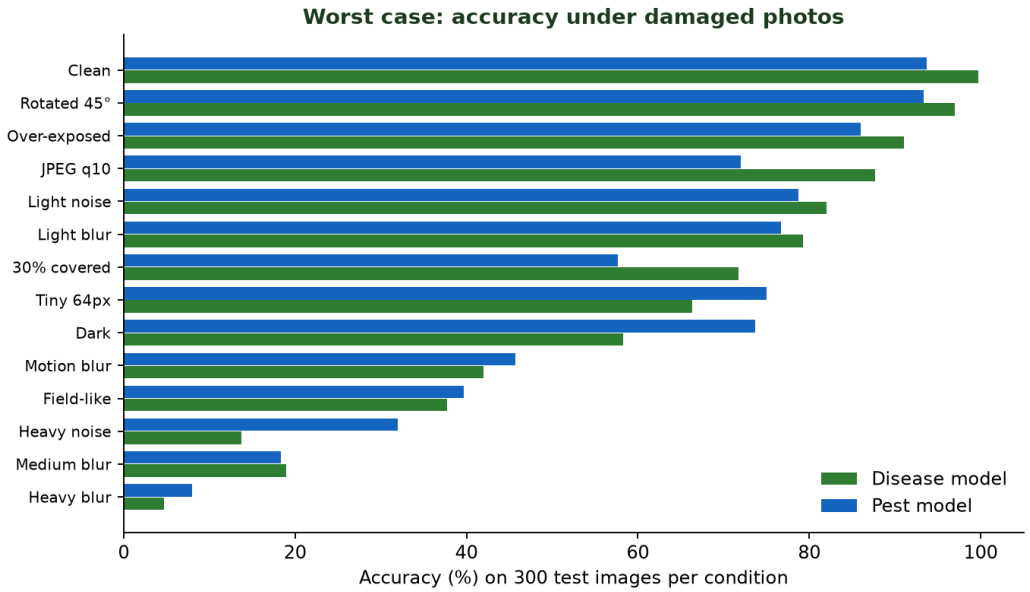


Figure 6. Accuracy under damaged photos (300 images per condition).

With a heavily blurred leaf photo the disease model is right only 4.7% of the time, and because it still answers, it is confidently wrong 71.7% of the time. The pest model is more cautious (it refuses most bad photos) but is right only 8% of the time. Off-topic inputs were accepted by mistake up to 99% of the time before the subject gate and a sharpness check were added. The product therefore asks for a clearer photo whenever confidence is low, shows results as advice, and records every diagnosis as accepted or uncertain for review.

12. Assistant, language and voice design

12.1 AgriBrain

AgriBrain is the single router for text, voice and phone calls. It owns no agricultural facts; it decides who answers and assembles the reply.

1. **NLU rules** detect the intent (weather, price, scheme, crop, yield, disease, pest, profit, calendar, help, greeting) and entities such as the crop name from per-language keyword lists. The English list is always added.
2. If a rule matches, the matching **tool** runs: it reads the database, a cache, a live source or an ML model.
3. If nothing matches and an LLM is configured, the model chooses an intent from the same list or answers open conversation, with the farmer's profile and the last 6 turns as context.
4. A message can contain up to two topics ("weather and rice price"); both are answered.
5. The reply is rendered from translated templates, saved with intent, handler and latency, and returned.

12.2 Language support

Eleven languages each have complete translation files: **382 keys in the app and 104 in the backend**, identical in every language (checked by script). The language is stored on the farmer's profile and used for screens, spoken replies and alerts. Fixed screens use curated translations; some generated text is translated through Gemini or Sarvam and cached in `translation_cache`.

12.3 Voice pipeline

```
microphone -> app records (max 30 s) -> POST /voice/session/{id}/utterance
  -> speech-to-text   (Sarvam, else Gemini lite, else Gemini flash)
  -> AgriBrain        (rules -> tool -> reply text in the spoken language)
  -> text-to-speech   (server voice, else the phone's own voice)
```

Check (20 Oct 2026, live providers)	Result
Typed question gives the right answer type, same language	33 of 33 (11 languages x 3 questions)
Voice command recognised correctly	17 of 17 testable; similarity 0.92 to 1.00 to what was said
Voice command gives the right answer type	17 of 17
Speech output (server voice)	Verified for English, Hindi, Telugu, Tamil, Kannada, Malayalam, Marathi, Bengali, Gujarati. Punjabi and Odia not verified: the free quota ran out
Latency	Typed answers under 0.1 s (prices about 4 s); voice turns 2.5 to 20 s

> **Free-tier fact:** Google's free tier allows only 10 speech-output requests per day per model. For real use the phone's own text-to-speech is the free path, and a paid Sarvam or Gemini plan is the quality path.

13. Core code walkthrough

The excerpts below are taken from the project and shortened (marked "abridged"). Each is followed by why it is written that way.

13.1 Password sign-in with lockout

backend/app/auth/service.py (abridged)

```
async def login_with_password(session, identifier, via, password, user_agent):
    user = await _find_farmer_user(session, identifier, via)
    if user is None or not user.password_hash:
        verify_password(_DUMMY_PASSWORD_HASH, password)   # same time as a real check
        raise Unauthorized("Wrong number, e-mail or password", code="invalid_credentials")
    _reject_if_locked(user)
    if not verify_password(user.password_hash, password):
        await _record_failed_login(session, user)
        raise Unauthorized("Wrong number, e-mail or password", code="invalid_credentials")
    return await _issue_tokens(session, user, user_agent)

async def _record_failed_login(session, user):
    user.failed_logins += 1
    if user.failed_logins >= MAX_FAILED_LOGINS:          # 5
        user.locked_until = utcnow() + timedelta(minutes=LOCK_MINUTES)   # 15
        user.failed_logins = 0
    await session.commit()   # the failure must survive the 401 that follows
```

Unknown accounts and wrong passwords return the same message and take the same time, so an attacker cannot tell which phone numbers are registered. After five failures the account locks for 15 minutes; the counter is committed before the 401 is returned so the failure is never lost.

13.2 Refresh-token rotation under a row lock

backend/app/auth/service.py

```
async def refresh(session, refresh_token, user_agent):
    auth_session = await session.scalar(
        select(AuthSession)
        .where(AuthSession.refresh_token_hash == hash_token(refresh_token))
        .with_for_update()
    )
    if auth_session is None or auth_session.revoked_at or auth_session.expires_at <= utcnow():
        raise Unauthorized("Session expired", code="session_expired")
    auth_session.revoked_at = utcnow()
    user = await session.get(User, auth_session.user_id)
    return await _issue_tokens(session, user, user_agent)
```

The `with_for_update()` lock means two simultaneous refreshes with the same token cannot both succeed. The old token is revoked as the new pair is issued.

13.3 Sliding-window rate limiter

backend/app/core/rate_limit.py (abridged)

```
class InMemoryWindow:
    def __init__(self):
        self._hits = defaultdict(deque)
        self._lock = Lock()

    def hit(self, key, limit, window_seconds) -> bool:
        now = time.monotonic()
        with self._lock:
            hits = self._hits[key]
            while hits and now - hits[0] > window_seconds:
                hits.popleft()                  # forget hits outside the window
            if len(hits) >= limit:
                return False
            hits.append(now)
            if len(self._hits) > 100_000:       # bound memory under abuse
                self._evict(now, window_seconds)
            return True
```

A deque of recent timestamps per key gives an exact sliding window. The lock makes it safe across threads and the size cap stops memory growing under attack. Authenticated callers are limited per user and anonymous ones per IP, so one noisy farmer does not block others behind the same mobile-carrier address.

13.4 Bounded CPU work for model inference

backend/app/core/concurrency.py

```
_limiter = None

def _get_limiter() -> CapacityLimiter:
    global _limiter
    if _limiter is None:
        _limiter = CapacityLimiter(get_settings().ml_max_concurrency)
    return _limiter

async def run_cpu_bound(func, *args, **kwargs):
    return await anyio.to_thread.run_sync(
        partial(func, *args, **kwargs), limiter=_get_limiter()
    )
```

The event loop handles I/O; CPU-heavy inference runs on worker threads but never more than the configured number at once. NumPy, XGBoost and ONNX Runtime release Python's GIL, so the threads give real parallelism, and excess requests wait instead of slowing everyone.

13.5 Crop ranking: model x region x rules

backend/app/crop/service.py (abridged)

```
tempered = {p.label: (p.probability + 1e-4) ** LIKELIHOOD_TEMPER for p in predictions}
mean = sum(tempered.values()) / len(tempered) if tempered else 1.0
for info in catalog.values():
    likelihood = tempered[info.slug] / mean if info.slug in tempered else 1.0
    regional = prior.get(info.slug, ABSENT_SHARE) if prior else 1.0   # area grown in this state
    adjustment = adjust(info, conditions)                                # agronomic rules
    score = likelihood * regional * adjustment.factor
```

This is the heart of the local-knowledge idea. The model's probability is a likelihood of the soil and climate readings, the regional share is a prior, and the rules scale the result. Crops the model does not cover (for example wheat) get a neutral likelihood of 1.0 and are ranked by region and rules, so the farmer still sees them.

13.6 Crop training split and honest tuning

ml/agrismart_ml/crop.py (abridged)

```
def _split(frame, seed):
    ...
    return labels, train_test_split(x, y, np.arange(len(frame)),
                                    test_size=0.2, stratify=y, random_state=seed)

def tune(frame, trials=30, seed=42, cv_folds=5):
    labels, (x_train, _, y_train, _, _, _) = _split(frame, seed)   # test rows are never used here
    cv = StratifiedKFold(n_splits=cv_folds, shuffle=True, random_state=seed)
    for trial in range(trials):
        choice = {k: v[rng.integers(len(v))] for k, v in SEARCH_SPACE.items()}
        scores = cross_validate(XGBClassifier(**params), x_train, y_train, cv=cv,
                                scoring=["neg_log_loss", "accuracy"])
```

The hyper-parameter search sees only the training rows, using cross-validated log-loss, so the 20% test rows stay untouched and the final score is not inflated by tuning.

13.7 Yield: time split and interval calibration

ml/agrismart_ml/yield_model.py (abridged)

```
def temporal_split(frame, test_years=3, valid_years=2):
    last = frame["crop_year"].max()
    test  = frame[frame["crop_year"] > last - test_years]
    valid = frame[(frame["crop_year"] > last - test_years - valid_years)
                  & (frame["crop_year"] <= last - test_years)]
    train = frame[frame["crop_year"] <= last - test_years - valid_years]
    return train, valid, test

def widen(log_q, scale):
    median = log_q[:, [1]]
    return median + (log_q - median) * scale      # stretch P10 and P90 away from the median
```

Splitting by year mimics forecasting. The band is widened until it covers the target share of the validation years, then coverage is reported honestly on the untouched test years.

13.8 Image training in two stages

ml/agrismart_ml/vision.py (abridged)

```
criterion = nn.CrossEntropyLoss(label_smoothing=config.label_smoothing)   # 0.1

def stage(name, params, lr, epochs):
    optimizer = torch.optim.AdamW(params, lr=lr, weight_decay=config.weight_decay)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs)
    ...

stage("head",     model.classifier.parameters(), config.head_lr,     config.head_epochs)       # 1e-3
stage("finetune", model.parameters(),            config.finetune_lr, config.finetune_epochs)   # 2e-4
```

Training only the new head first protects the pre-trained features from large random updates. The cosine schedule lowers the learning rate smoothly, and label smoothing discourages over-confidence, which matters because the app relies on confidence to decide when to answer.

13.9 Assistant core

backend/app/assistant/brain.py (abridged)

```
async def handle(self, session, ctx, text, session_id):
    chat = await self._chat_session(session, ctx, session_id)
    with Timer() as timer:
        intent, result = await self._route(session, ctx, chat, text)   # rules, then tool or LLM
    metrics.inc("assistant_intents", f"{intent.value}:{result.handled_by}")
    session.add_all([
        ChatMessage(session_id=chat.id, role="farmer", text=text[:4000], intent=intent.value),
        ChatMessage(session_id=chat.id, role="assistant", text=result.text[:4000],
                    intent=intent.value, handled_by=result.handled_by, latency_ms=timer.ms),
    ])
    await session.commit()
    return AssistantReply(session_id=chat.id, intent=intent.value, handled_by=result.handled_by,
                          language=ctx.language, text=result.text, data=result.data, action=result.action)
```

Every question, from any channel, goes through this one function: route, answer, record intent, handler and latency, then reply. The saved latency and handler (rule, tool or model) are what the admin dashboard uses to see how the assistant behaves.

13.10 Speech recognition model order

backend/app/core/config.py and voice/providers/gemini.py

```
# config.py
gemini_asr_models: list[str] = ["gemini-flash-lite-latest", "gemini-flash-latest"]

# providers/gemini.py
self._asr_models = settings.gemini_asr_models
result = await self._generate("asr", self._asr_models, payload, timeout=12)
```

During testing the full Gemini flash model stalled on audio for 30 s or more, using up the 25 s interactive deadline before the fast model was ever tried. Speech recognition now has its own list with the fast lite model first and a 12 s limit per model, which brought voice replies to about 6 to 8 seconds.

13.11 Client sign-in call

mobile/src/app/login.tsx (abridged)

```
const submitSignIn = () =>
  run(async () => {
    await signIn(await api<TokenPair>('/auth/password-login',
      { method: 'POST', body: { ...identity, password }, auth: false }));
  });

const submitCreate = () =>
  run(async () => {
    if (password !== confirm) throw new ApiError('password_mismatch', '', 0);
    await signIn(await api<TokenPair>('/auth/register',
      { method: 'POST', body: { ...identity, password, name: name.trim(), language }, auth: false }));
  });
```

The app only talks to the AgriSmart API. The same two calls work for a phone or the web build, so a password created on one is valid on the other. Tokens go to the secure store through `signIn`.

13.12 Security headers middleware

backend/app/main.py (abridged)

```
async def send_wrapper(message):
    if message["type"] == "http.response.start":
        message.setdefault("headers", []).extend([
            (b"x-content-type-options", b"nosniff"),
            (b"x-frame-options", b"DENY"),
            (b"referrer-policy", b"no-referrer"),
            (b"strict-transport-security", b"max-age=31536000; includeSubDomains"),
        ])
    await send(message)
```

A tiny ASGI wrapper adds the headers to every response without touching individual routes.

14. Testing and quality assurance

Check	Result
Backend automated tests	234 passed (API, password sign-in, voice providers, telephony, NLU, markets, schemes, configuration)
ML contract tests	6 passed (training, ONNX export, serving round trip)
Mobile app	TypeScript type check and lint clean
Admin website	Type check clean; production build succeeds
Live API and PostgreSQL	Register, restart the API, sign in again with the same password; password stored as an Argon2id hash
Languages and voice	33 of 33 typed answers; 17 of 17 testable voice commands (section 12.3)
Unseen-data tests	Scripts in `ml/checks/` re-run all the generalisation checks (section 11.4)
Load test	0% errors from 100 to 1,000 simulated farmers (section 9.3)

A live check also found and fixed real defects: the market price endpoint returned an error whenever the live Agmarknet provider was selected (an undefined variable), voice recognition failed because the first model stalled, and the yield error metric was mislabelled. Each fix has a regression test or a documented measurement.

15. Deployment and operation

Task	How
Run everything on Windows	`powershell -ExecutionPolicy Bypass -File .\start-all.ps1` starts the database, API, worker, admin and Expo, and prints the QR code
Backend only	`cd backend`, create a virtualenv, `pip install -e .`, copy `.env.example` to `.env`, `alembic upgrade head`, `python -m app.cli seed`, `uvicorn app.main:app`
Create an admin	`python -m app.cli create-admin you@example.com` (or `set-password` to reset)
Farmer app on a phone	`cd mobile`, set `EXPO_PUBLIC_API_URL` to the PC's Wi-Fi address, `npx expo start --lan`, scan the QR code with Expo Go
Check in mobile view	`npx expo start --web`, open it in Chrome, press F12 then Ctrl+Shift+M and pick a phone
Android APK	`eas build -p android --profile preview` after setting the API address in `mobile/eas.json`; install the downloaded .apk on the phone
Admin website	`cd admin`, set `AGRISMART_API_URL`, `npm run build`, `npx next start`; or deploy to Vercel
Production stack	`docker compose -f deploy/docker-compose.yml up -d --build`, then `--scale api=4` to add instances
Train the models	Download the Kaggle datasets, run `agrismart_ml.datasets`, then the four notebooks; register with `python -m app.cli register-model <name> <version> --activate`

16. Limitations and future work

16.1 Known limitations

- **Image models are weak on bad photos** (down to 4.7% accuracy on heavy blur) and field-photo accuracy has not been measured with real field images.
- **Disease classes** include many non-Indian crops; rice and wheat diseases are not covered.
- **Yield model** does not respond to rainfall, fertiliser or pesticide, only to crop, state and year, and does not transfer to unseen states or crops.
- **Crop model** loses accuracy quickly when soil values are estimated, and has no "unknown" answer on its own.
- **Free voice output** is limited to 10 Google requests per day per model; the phone's own voice depends on the installed languages. Punjabi and Odia speech output is not verified.
- **Unverified registration:** the first person to register a phone number owns the account.
- **Load figures** come from one laptop; the in-process rate limiter is per instance (the OTP limit and password lockout are global because they are in PostgreSQL).
- **Translations** should be reviewed by native speakers, especially safety advice about pesticides.

16.2 Recommended next steps

1. Collect and label real field photos; retrain with a split by source photo; add the Indian staples (rice, wheat, sugarcane).
2. Add an input-range check and an "uncertain" output to the crop model; collect real soil-test data to replace the idealised dataset.
3. Replace or extend the yield model with district-level weather and irrigation data.
4. Move the rate limiter to Redis if exact global limits are needed; add PgBouncer for large fleets.
5. Add OTP verification at registration once an SMS or e-mail service is chosen.
6. Evaluate on-device speech and a small offline model for farmers with no data.

17. Conclusion

AgriSmart shows that a trustworthy farmer assistant can be built with free and open technology: a stateless FastAPI service on PostgreSQL, a single cross-platform app, ONNX models that run on a CPU, and a voice chain that degrades gracefully from paid to free. Its main contribution is not a single accuracy number but a design that **measures its own limits and works around them**: confidence gates, a photo-quality check, regional knowledge on top of models, published worst cases, and a database-backed sign-in that needs no paid service. The remaining work is mostly data, not code: real field photos, real soil tests and native-speaker review.

Appendix A. Project folder map

Path	Contents
`backend/app/`	FastAPI code by feature: `auth`, `assistant`, `voice`, `crop`, `yield_prediction`, `disease`, `pest`, `market`, `schemes`, `weather`, `notification`, `support`, `admin`, `core`
`backend/migrations/`	Alembic database migrations
`backend/tests/`	234 automated tests
`backend/loadtest/`	Load test and the language and voice check
`mobile/src/`	App screens (`app/`), API client, auth, i18n (11 languages), voice, components
`admin/src/`	Next.js dashboard pages and the server-side API helpers
`ml/`	Notebooks, `agrismart_ml` package, evaluation scripts, `checks/` (unseen-data tests)
`deploy/`	Docker Compose, nginx configuration, free-hosting guide
`legacy/`	The earlier prototype, kept for reference only

Appendix B. Glossary

Term	Meaning
Stateless API	A server that keeps no per-user data in memory, so any copy can answer any request
Load balancer	A front door that spreads requests across several copies of the server
least_conn	A balancing rule that sends each request to the server with the fewest active requests
Argon2id	A deliberately slow password-hashing method resistant to brute-force guessing
JWT	A signed token proving who the caller is for a short time
Refresh token rotation	Each refresh issues a new token and cancels the old one
XGBoost	A method that builds many small decision trees, each fixing the previous errors
Quantile regression	Predicting a range (for example the 10th and 90th percentile) instead of one number
Transfer learning	Starting from a network already trained on millions of photos and adapting it
ONNX / int8	A portable model format / numbers stored in 8 bits to make a model smaller and faster
Cross-validation	Repeatedly training on part of the data and checking on the rest to estimate accuracy
Leakage	When test data secretly resembles training data, inflating the score
NLU	Natural-language understanding: working out what a question is asking
TTS / ASR	Text-to-speech (speaking) and automatic speech recognition (listening)
p50 / p95 / p99	Response time that 50%, 95% and 99% of requests beat